MALWARE AND PHISHING DETECTION FOR MOBILE SECURITY

A PROJECT REPORT

Submitted by

SYED ARSHATH S

(2023178050)

submitted to the Faculty of

INFORMATION AND COMMUNICATION ENGINEERING

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI 600 025

May 2025

ANNA UNIVERSITY

CHENNAI - 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **"MALWARE AND PHISHING DETECTION FOR MOBILE SECURITY"** is the bonafide work of **SYED ARSHATH S (2023178050)** who carried out project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: CHENNAI

DATE:

Dr. P. GEETHA
ASSOCIATE PROFESSOR
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI 600025

COUNTERSIGNED

Dr. S. SWAMYNATHAN
HEAD OF THE DEPARTMENT
DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY
CHENNAI 600025

ABSTRACT

As mobile devices become integral to daily life, they face an escalating of cyber threats, from deceptive scams to insidious malware and phishing attacks. This mobile application emerges as a comprehensive security, designed to empower users with proactive protection in an increasingly vulnerable digital landscape. Seamlessly integrating three core functionalities, the app offers robust detection of fraudulent messages, identifies malicious software with actionable response options, and blocks phishing attempts to safeguard sensitive data. Built with a user-centric approach, it prioritizes intuitive navigation and real-time alerts, ensuring both novice and tech-savvy individuals can secure their devices effortlessly. By addressing diverse threats through a unified platform, this solution not only enhances mobile safety but also sets a new standard for integrated cybersecurity tools.

The application brings together different types of protection into a single system, allowing users to address multiple threats without complexity. It runs in the background, keeping devices functional while maintaining security. It uses a BERT model for scam detection. The BERT model was fine-tuned using a collective dataset from Kaggle. The Androzoo dataset, known for its vast collection of Android malware samples, was used to train XGBoost for fast, accurate malware prediction. Androzoo is widely recognized for its comprehensive and diverse set of malware data, making it an ideal resource for training models aimed at detecting mobile threats. The app also includes a VPN-based phishing protection module that blocks malicious websites in real time.

திட்ட சுருக்கம்

அலைபேசிச் சாதனங்கள் இன்றைய வாழ்வில் அத்தியாவசியப் பங்கு வகிக்கின்றன. இருப்பினும், அவை பெருகிவரும் இணைய அச்சுறுத்தல்களான ஏமாற்று வேலைகள், தீங்குநிரல்கள் மற்றும் ஃபிஷிங் தாக்குதல்களுக்கு ஆளாகின்றன. இத்தகைய ஆபத்துகளிலிருந்து பயனர்களைப் பாதுகாக்க, இந்த அலைபேசிச் செயலி விரிவான பாதுகாப்புத் தீர்வாக உருவாக்கப்பட்டுள்ளது. மூன்று முக்கியச் செயல்பாடுகளைத் தடையின்றி ஒருங்கிணைத்து, இந்தச் செயலி மோசடிச் செய்திகளைத் துல்லியமாகக் கண்டறிதல், தீங்குநிரல்களை அடையாளம் கண்டு உரிய நடவடிக்கைகளை மேற்கொள்ள வழிகாட்டுதல், மற்றும் முக்கியமான தரவுகளைப் பாதுகாக்க ஃபிஷிங் முயற்சிகளைத் தடுத்தல் ஆகியவற்றை வழங்குகிறது. பயனர் மைய அணுகுமுறையுடன் வடிவமைக்கப்பட்ட இது, எளிமையான வழிசெலுத்தல் மற்றும் உடனடி அறிவிப்புகளுக்கு முக்கியத்துவம் அளிக்கிறது. இதன் மூலம் அனைவரும் தங்கள் சாதனங்களைப் பாதுகாப்பாக வைத்திருக்க முடியும். பல்வேறு அச்சுறுத்தல்களை ஒரே தளத்தில் கையாள்வதன் மூலம், இந்தத் தீர்வு அலைபேசிப் பாதுகாப்பை மேம்படுத்துவதுடன், ஒருங்கிணைந்த இணையப் பாதுகாப்புக் கருவிகளுக்கான புதிய தரத்தையும் அமைக்கிறது. இந்தச் செயலி பல்வேறு பாதுகாப்பு முறைகளை ஒரே அமைப்பில் ஒருங்கிணைத்து, பயனர்கள் பல அச்சுறுத்தல்களை எளிதாக எதிர்கொள்ள உதவுகிறது. இது பின்னணியில் இயங்கி, சாதனத்தின் செயல்பாட்டிற்குப் பாதிப்பின்றிப் பாதுகாப்பை உறுதி செய்கிறது. ஏமாற்று வேலைகளைக் கண்டறிய ஒரு குறிப்பிட்ட மாதிரியைப் பயன்படுத்துகிறது. இந்த மாதிரியானது ஒரு குறிப்பிட்ட தரவுத்தொகுப்பைப் பயன்படுத்திப் பயிற்றுவிக்கப்பட்டது. ஆண்ட்ராய்டு தீங்குநிரல் மாதிரிகளின் பெரிய தொகுப்புக்கு அறியப்பட்ட ஒரு தரவுத்தொகுப்பு, விரைவான மற்றும் துல்லியமான தீங்குநிரல் கணிப்புக்காக ஒரு வகை அல்காரிதமிற்குப் பயிற்சி அளிக்கப் பயன்படுத்தப்பட்டது. இந்தத் தரவுத்தொகுப்பு அதன் விரிவான மற்றும் பல்வேறு தீங்குநிரல் தரவுகளுக்குப் பரவலாக அங்கீகரிக்கப்பட்டுள்ளது. இது அலைபேசி அச்சுறுத்தல்களைக் கண்டறியும் மாதிரிகளுக்குப் பயிற்சி அளிக்கச் சிறந்த ஆதாரமாக அமைகிறது. இந்தச் செயலியானது நிகழ்நேரத்தில் தீங்கிழைக்கும் வலைத்தளங்களைத் தடுக்கும் ஒரு குறிப்பிட்ட வகை பாதுகாப்புத் தொகுதியையும் கொண்டுள்ளது.

ACKNOWLEDGEMENT

It is my privilege to express my sincere thanks to my project guide, **Dr.P.Geetha**, Associate Professor, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, Chennai, for her genuine interest, inspiring guidance, constant encouragement, and unwavering support throughout all stages of my project, helping bring this report to completion.

I also extend my sincere thanks to **Dr. S. Swamynathan**, Professor & Head, Department of Information Science and Technology, College of Engineering, Guindy Campus, Anna University, Chennai, for his constant support.

I would like to express my sincere thanks to the project committee members, **Dr.T.Mala** Professor, **Dr.N.Thangaraj** Associate Professor, **Dr.E.Uma** Associate Professor, and **Ms.R.L.Jasmine** Teaching Fellow, Department of Information Science and Technology, College of Engineering, Guindy Campus, Anna University, Chennai, for their valuable suggestions throughout the duration of my project.

SYED ARSHATH S

2023178050

TABLE OF CONTENTS

	ABSTRACT	iii
	ABSTRACT (TAMIL)	iv
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 OBJECTIVE	2
	1.3 PROBLEM STATEMENT	2
	1.4 SOLUTION OVERVIEW	3
	1.5 ORGANIZATION OF THE REPORT	3
2	**LITERATURE SURVEY**	**5**
	2.1 SCAM DETECTION	5
	2.2 MALWARE DETECTION	6
	2.3 PHISHING DETECTION	8
	2.4 LIMITATIONS OF EXISTING WORKS	9
	2.5 SUMMARY	9
3	**DESIGN OF PROPOSED SYSTEM**	**10**
	3.1 TECHNICAL ARCHITECTURE	10
	3.2 DATA PREPROCESSING	10
	3.2.1 Scam Detection Dataset	12
	3.2.2 Malware Detection Dataset	12
	3.2.3 Phishing Detection Dataset	12
	3.3 SCAM DETECTION MODEL	13
	3.3.1 Dataset Description	13
	3.3.2 Model Fine-Tuning	13
	3.3.3 Application Integration	14
	3.4 Malware Detection Model	15
	3.4.1 Dataset and Feature Preparation	15
	3.4.2 Quantum Encoding with Qiskit	16
	3.4.3 Training and Environment	16

3.4.4 Application Integration 17
3.5 Phishing Detection Agent 18
3.5.1 Dataset and Preprocessing 18
3.5.2 Fine-Tuning GPT-2 19
3.5.3 Agentic Functionality and Domain Analysis 20
3.5.4 Training and Inference Environment 20
3.5.5 Application Integration 21
3.6 SUMMARY 22

4 IMPLEMENTATION OF PROPOSED SYSTEM 23
4.1 TOOLS AND LIBRARIES USED 23
4.2 BERT FINETUNING FOR SCAM DETECTION 24
4.3 PREPROCESSING DATASET TO QUANTUM STATE 25
4.4 HYBRID QUANTUM XGBOOST TRAINING 26
4.5 FINE-TUNING GPT-2 FOR PHISHING DETECTION 27
4.6 PHISHING DETECTION AGENT IMPLEMENTATION 28

5 RESULTS AND PREFORMANCE ANALYSIS 30
5.1 RESULTS 30
5.1.1 DATASET USED FOR TRAINING 30
5.1.2 MODEL PREDICTIONS IN MOBILE INTERFACE 31
5.2 PERFORMANCE ANALYSIS 40
5.2.1 SCAM DETECTION 40
5.2.2 MALWARE DETECTION 43
5.2.3 PHISHING DETECTION 46

6 CONCLUSION AND FUTURE WORK 49
6.1 CONCLUSION 49
6.2 FUTURE WORK 49

REFERENCES 50

LIST OF TABLES

5.1 Classification Report for Scam Detection 41
5.2 Classification Report for Hybrid Quantum-Classical Malware Detection 45
5.3 Classification Report for Classical Malware Detection 45
5.4 Classification Report for Phishing Detection (93000 Samples) 48

LIST OF FIGURES

3.1 Technical Architecture of the Proposed System 11
3.2 BERT Model Fine-Tuning Workflow for Scam Detection 14
3.3 Integration of Scam Detection Model in Mobile App Workflow 15
3.4 Feature Extraction and Quantum Encoding 17
3.5 Malware Detection and App Flagging Workflow 17
3.6 Phishing Agent Workflow 21
3.7 Phishing Detection From User Interface 22

5.1 Mobile App Initial Screens 32
5.2 Notification for Detected Scam Message 34
5.3 Detailed View of the Flagged Scam Message 34
5.4 Turn on Malware Protection from the App Settings 36
5.5 User Installs a New Application which Triggers a Scan 36
5.6 Real-Time Scan Result shown as a Notification (Malware or Benign) 37
5.7 Inbuilt WebView used for Browsing within the App 39
5.8 Alert Screen Displayed when a Phishing URL is Blocked 39
5.9 Phishing Detection Log Output Captured from Logcat 40
5.10 Confusion Matrix for Scam Detection 42
5.11 Accuracy, Precision, and Recall Curves for Scam Detection over Epochs (Training/Validation) 42
5.12 ROC Curve for Scam Detection (AUC = 0.90) 43
5.13 Performance Metrics (Accuracy, Precision, Recall) for Quantum Hybrid Malware Detection 44
5.14 Performance Metrics (Accuracy, Precision, Recall) for Classical Malware Detection 44
5.15 Confusion Matrix for Hybrid Malware Detection 46
5.16 Confusion Matrix for Classical Malware Detection 46
5.17 Overall Performance Comparison of Hybrid and Classical Malware Detection Models 46
5.18 Confusion Matrix for Phishing Detection 48
5.19 Accuracy, Precision, and Recall Curves for Phishing Detection over Epochs (Training/Validation) 48

LIST OF ABBREVIATIONS

APK	Android Package Kit
BERT	Bidirectional Encoder Representations from Transformers
DEX	Dalvik Executable
GPT-2	Generative Pretrained Transformer-2
HTML	HyperText Markup Language
LLM	Large Language Model
npy	NumPy binary file format
NLP	Natural Language Processing
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
XGBoost	eXtreme Gradient Boosting

CHAPTER 1

INTRODUCTION

Scams, malicious apps (malware) and phishing websites are significant threats to online safety. Scams often appear as deceptive messages that promise money or job opportunities, tricking individuals into revealing personal information or sending money. Malware can be hidden in apps or links, stealing sensitive data, or locking devices until a ransom is paid. Phishing attacks deceive users into sharing passwords or banking information masquerading as trusted organizations through emails, text messages, or fraudulent websites. Early detection of these threats is crucial to protecting personal data and ensuring online security.

This project aims to provide a comprehensive security solution for mobile devices, addressing the growing range of cyber threats that users face daily. In an increasingly digital world, users are vulnerable to scams, malware, and phishing attacks that can compromise their personal information and privacy. The project focuses on offering an easy and seamless platform that works in the background, protecting users without interrupting their experience. By combining various protective measures, it ensures that mobile devices remain secure, providing peace of mind to users while they navigate the digital landscape.

1.1 BACKGROUND

This project focuses on developing a mobile application that helps users stay safe online by detecting scams, malware, and phishing threats. It uses a fine-tuned BERT model for scam detection and XGBoost for fast,

accurate malware prediction. The app also includes a VPN-based phishing protection module that blocks malicious websites in real time, providing users with continuous security through instant alerts and proactive threat prevention.

1.2 OBJECTIVE

- To implement a smart scam detection system using a fine-tuned BERT model that analyzes messages and sends alerts.
- To develop a malware detection model using XGBoost, scanning installed apps and providing fast, accurate threat predictions.
- To prevent phishing attacks by analyzing URLs in real-time through an in-app WebView browser, blocking access to malicious websites before the user proceeds.
- To provide instant alerts and notifications whenever a potential threat is detected.

1.3 PROBLEM STATEMENT

The proposed mobile security application integrates several advanced machine learning models to provide a comprehensive and proactive defense against cyber threats. The application leverages fine-tuned BERT for scam detection, Quantum XGBoost for malware analysis, and GPT-2 for phishing prevention. Each of these models is designed to work seamlessly together to enhance the app's security capabilities.

The application includes an in-app WebView browser that ensures secure browsing by analyzing each URL in real-time. Before any site is fully loaded, the system evaluates the URL to detect potential threats, such as

phishing attempts, malicious websites, or malware downloads. If a threat is detected, the app proactively blocks the website from loading, preventing the user from being exposed to harmful content.

1.4 SOLUTION OVERVIEW

This project proposes a mobile application that helps users stay protected from cyber threats through real-time detection and alerts. It uses a BERT-based model to analyze messages and identify potential scams, sending immediate notifications when suspicious content is found. A malware detection module scans installed apps to detect harmful software. For phishing protection, the app uses a VPN-based approach to intercept and block malicious URLs while browsing. By providing timely alerts and preventive measures, the application ensures users are informed and protected from online threats.

1.5 ORGANIZATION OF THE REPORT

This thesis is organized into 6 chapters, describing each part of the project with detailed illustrations and system design diagrams. The chapters are as follows:

Chapter 2: explains the literature survey details of the existing systems with their methodologies, advantages and disadvantages.

Chapter 3: consists of the system design of the project with the description of the modules used in the project.

Chapter 4: explains in detail about the implementation of each module in the system design.

Chapter 5: provide the details of results and the performance analysis of the proposed system with the state of art method.

Chapter 6: concludes the project report. The future works that can be done to improve the proposed system is also given.

CHAPTER 2

LITERATURE SURVEY

This chapter explains the literature survey done in the existing system and the analyses of the problem statement and issues in the existing system.

2.1 SCAM DETECTION

[1] Utilized Naïve Bayes algorithm for spam classification in SMS and emails, leveraging probabilistic modeling to analyze word distributions. The model is trained on labeled datasets and applies Bayes' theorem to determine the likelihood of messages being spam. This approach is computationally efficient and suitable for real-time filtering. However, its assumption of feature independence limits its ability to detect sophisticated scams that rely on contextual deception and evolving language patterns.

[2] Compared various machine learning classifiers, including decision trees, support vector machines (SVM), and deep learning models, to determine the most effective spam detection approach. Feature engineering techniques like TF-IDF and word embeddings are explored to enhance classification accuracy. While this provides insights into model performance, the study does not conclude which classifier is optimal. Additionally, it lacks discussion on real-time adaptability and computational efficiency, making practical implementation unclear.

2.2 MALWARE DETECTION

[3] Explored the use of quantum machine learning for Android malware detection, comparing classical deep learning models (AlexNet, MobileNet, EfficientNet, VGG16, VGG19) with quantum models (hybrid and fully quantum). A dataset of 8446 Android apps was converted into grayscale images for classification. The best performing model was a custom Standard-CNN with 97% accuracy, while the hybrid quantum model achieved 90.5% accuracy, outperforming some classical models. The fully quantum model had lower performance (41.3%) due to hardware limitations. Grad-CAM was applied to provide explainability by highlighting important image regions. This work represents one of the first attempts to apply quantum machine learning to malware detection and provide explainability for quantum models, showing potential for future improvements as quantum computing advances.

[4] This paper presents a quantum-enhanced hybrid interaction network (QEHIN) for detecting real and fake Android applications to enhance user privacy. The proposed QEHIN model achieves superior performance compared to existing methods, including DroidEncoder, RealMalSol, GHGDroid, AAMD-OELAC, MAD-NET, and BAI-AMDC, as well as popular antivirus tools like AVG, Norton, and Avast. Evaluated on a dataset of Android apps, the QEHIN method demonstrates high accuracy (98.86%), precision (98.78%), recall (98.82%), and specificity (97.91%), with low false positive (0.0148%) and false negative rates (0.0942%). The model incorporates a quantum embedding layer, a quantum hybrid interaction network (QHIN), and a quantum deep neural network (QDNN), which collectively contribute to its improved performance. An ablation study confirms the importance of each component, showing significant drops in performance when any module is removed. Additionally, the QEHIN technique exhibits lower training (120s) and inference times (0.5s) compared to other methods, ensuring scalability.

The authors also validate their approach on a large-scale MobileRec dataset, which includes 19.3 million user interactions across 10,000 apps, further demonstrating the model's generalizability. This work highlights the potential of quantum computing in enhancing malware detection systems by leveraging quantum embeddings and hybrid architectures to process complex feature interactions efficiently.

Quantum machine learning is applied[5] for explainable malware detection using a dataset of 14,226 malware files and 1,832 trusted samples from the Malevis webpage, converted into grayscale images for classification. Classical deep learning models such as AlexNet, VGG16, and EfficientNet are compared with quantum-enhanced models, including a Hybrid Quantum Convolutional Neural Network (Hybrid-QCNN). The best performance is achieved by a custom Standard-CNN with 97% accuracy, while the Hybrid-QCNN reaches 90.5%, outperforming some classical models but not fully utilizing quantum advantages. A fully quantum model performs significantly lower (41.3%) due to hardware limitations. Grad-CAM is used to highlight critical regions in input images, improving explainability and assisting in identifying malware families or variants. The integration of quantum computing in malware detection shows promising potential for future advancements, while also revealing current challenges such as scalability and computational efficiency.

These studies highlight the potential of machine learning-based approaches in Android malware detection, with ensemble methods and hybrid frameworks proving particularly effective. However, limitations related to computational overhead, static feature dependence, and real-time scalability still need to be addressed to create robust malware detection systems.

2.3 PHISHING DETECTION

[6] Applied BERT to phishing email detection by analyzing email content and metadata, improving accuracy through contextual understanding. Another method enhances phishing URL detection by integrating BERT with URL feature extraction, allowing the model to identify subtle phishing patterns in web addresses. A lightweight variation uses Tiny-BERT to reduce computational overhead while maintaining detection efficiency. Hybrid techniques combine BERT with machine learning classifiers for better performance. Despite their strengths, these methods have limitations. BERT models require high computational power, making real-time deployment challenging. While Tiny-BERT improves efficiency, it may lose some accuracy. Static training data limits adaptability to evolving phishing tactics, and hybrid approaches introduce additional model complexity. Continuous updates and optimized architectures are needed to address these challenges.

Another study investigated[7] the application of large language models (LLMs) in detecting phishing attempts. The authors explore how LLMs, known for their advanced natural language understanding, can be leveraged to identify phishing content in emails and messages. While LLMs offer improved detection capabilities, the study notes challenges related to computational resource requirements and potential biases in the training data, which could affect the models' reliability in diverse real-world scenarios.

[8]Further research focuses on utilizing machine learning techniques to detect phishing attacks by analyzing HTTP header information. By extracting features from HTTP headers, the proposed model aims to identify malicious activities without relying on the content of the web pages. However, the study highlights limitations, such as the model's dependency on the availability and quality of header information, which can be manipulated by sophisticated attackers, potentially leading to evasion of detection mechanisms.

2.4 LIMITATIONS OF EXISTING WORKS

- High computational overhead, making real-time deployment difficult.
- Poor adaptability to evolving phishing, malware, and scam tactics.
- Scalability issues with high memory usage and slow processing.

2.5 SUMMARY

This chapter focuses on the literature survey of existing systems for scam detection, malware detection, and phishing detection. It analyzes the problem statements and issues present in these existing systems. The chapter details various approaches used in previous works, such as Naive Bayes for SMS and email spam classification, comparison of machine learning classifiers for spam detection, Natural Language Processing (NLP) techniques for detecting scams in SMS and call transcripts, and the application of quantum machine learning for Android malware detection. It also discusses the use of BERT and other machine learning techniques for phishing detection by analyzing content, metadata, and URL features. The chapter concludes by highlighting the limitations of these existing works, including high computational overhead, poor adaptability to evolving threats, and scalability issues.

CHAPTER 3

DESIGN OF PROPOSED SYSTEM

This chapter consists of system design with the technical architecture and the module architecture and the description of the modules used in the project.

3.1 TECHNICAL ARCHITECTURE

The technical architecture diagram of the proposed system, shown in Figure 3.1, depicts these modules and illustrates the overall system flow and their interaction.

- Scam detection
- Malware detection
- Phishing detection

3.2 DATA PREPROCESSING

Data collection refers to the process of obtaining, measuring, and analyzing data from various sources in order to prepare the dataset for model training. In this project, datasets for scam detection, malware detection, and phishing detection were collected from reliable sources and further preprocessed to ensure consistency, balance, and relevance for the training phase.

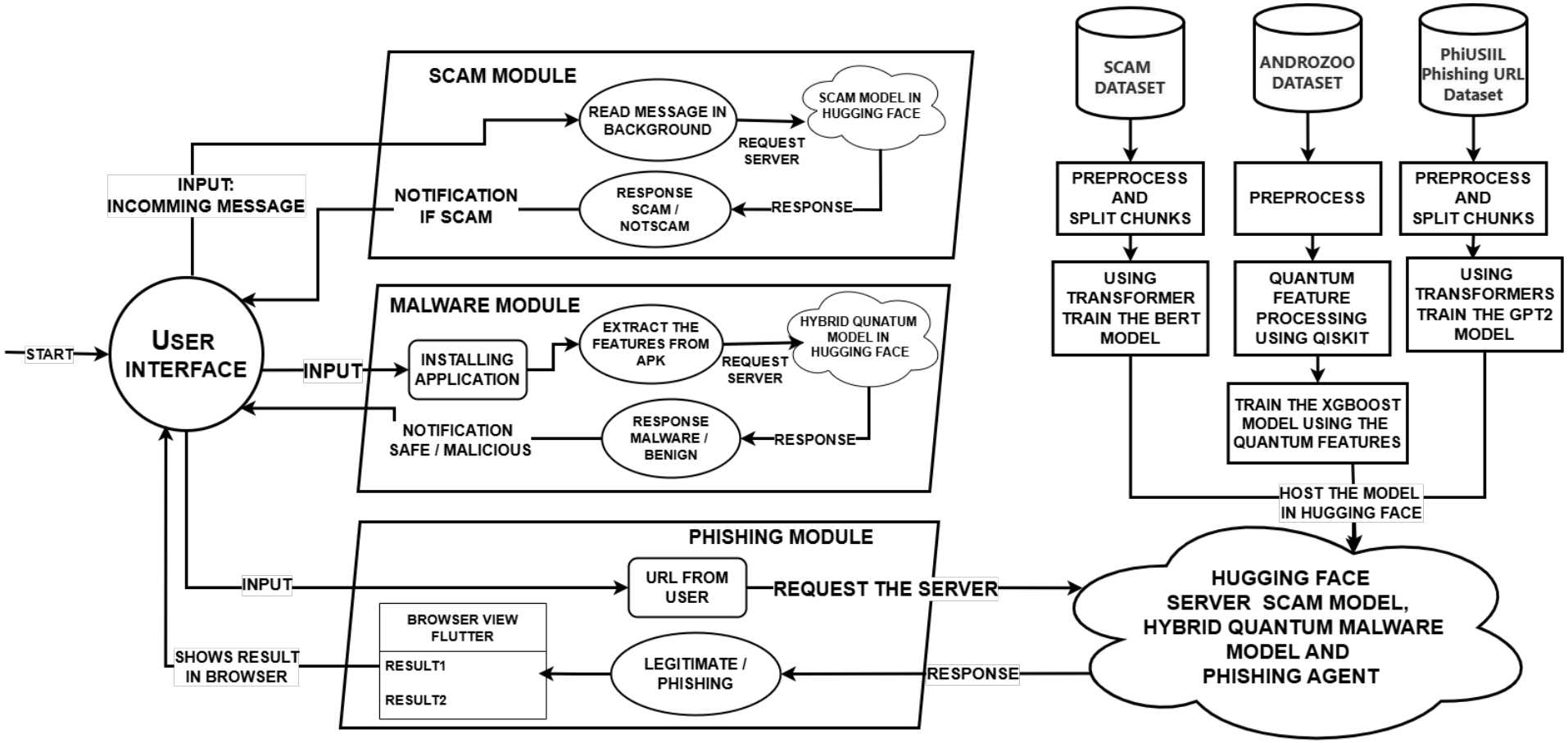


Figure 3.1: Technical Architecture of the Proposed System

3.2.1 Scam Detection Dataset

The dataset for scam detection was collected from Kaggle, containing real-world examples of scam messages. To enhance the dataset size and improve generalization, data augmentation techniques were applied. This included text paraphrasing, synonym replacement, and noise injection to create a diverse set of scam and non-scam samples.

3.2.2 Malware Detection Dataset

For malware detection, the AndroZoo dataset[9] was used, which contains 25 millions of Android application samples. The dataset includes both benign and malicious APKs, categorized into different malware families. Feature extraction was performed on these samples using python qiskit package to transform the main features to quantum state. Used features apk_size, dex_size, apk_dex_ratio as 3 qubits for quantum circuits in 16 shots and 3 classical features pkg_name, markets, vercode and saved as .npy file for training a hybrid qxgboost for android malware detection.

3.2.3 Phishing Detection Dataset

The phishing detection module utilizes the PhiUSIIL Phishing URL dataset[10], Dataset is a substantial dataset comprising 134,850 legitimate and 100,945 phishing URLs. Most of the URLs we analyzed, while constructing the dataset, are the latest URLs. Features are extracted from the source code of the webpage and URL. Features such as CharContinuationRate, URLTitleMatchScore, URLCharProb, and TLDLegitimateProb are derived from existing features. The URLs were analyzed for lexical features, domain characteristics, and metadata. After cleaning and removing duplicates, the dataset was split for training and evaluation of phishing detection model.

3.3 SCAM DETECTION MODEL

The scam detection module leverages the BERT (Bidirectional Encoder Representations from Transformers) language model, a pre-trained transformer-based model developed by Google for a wide range of NLP tasks. In this project, BERT was fine-tuned using a custom scam detection dataset collected and generated through data augmentation techniques.

3.3.1 Dataset Description

The dataset contains text messages labeled as either **Scam** or **Not Scam**, along with associated categories (e.g., Loan, Job Offer, Lottery) and corresponding safe-response messages. An example entry from the dataset is shown below:

Listing 3.1: Sample Scam Detection Data

```
{
   "text": "Congratulations! You have been selected for \
   a special Loan reward. Claim now!",
   "category": "Loan",
   "response": "This is a fraudulent attempt to steal \
   your details. Ignore and report immediately.",
   "label": "Scam"
}
```

3.3.2 Model Fine-Tuning

The BERT model was fine-tuned using the scam detection dataset. The process involved tokenizing the message text, encoding the labels, and training the model using a binary cross-entropy loss. The model was trained

to classify input messages into scam or non-scam categories with high accuracy. The workflow for this fine-tuning process is illustrated in Figure 3.2.

The model was trained using a GPU environment, and after achieving satisfactory performance, it was saved and uploaded to Hugging Face Spaces for real-time access.

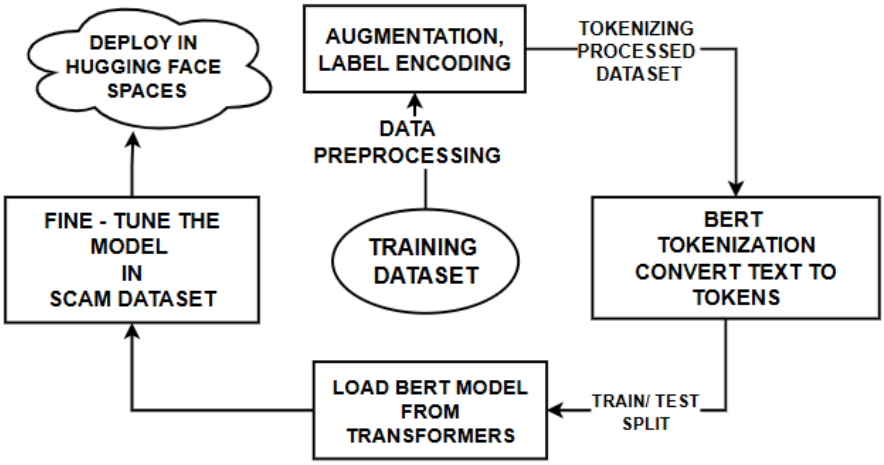


Figure 3.2: BERT Model Fine-Tuning Workflow for Scam Detection

3.3.3 Application Integration

Once the model was deployed on Hugging Face, it was integrated into the mobile security application. The application continuously monitors incoming SMS messages using background services. Each new message is automatically sent to the scam detection model via API. Based on the model's prediction, if the message is identified as a scam, it is flagged and a notification alert is sent to the user. If it is not a scam, the message is ignored by the scanner.

Flagged messages are stored in a local database for user review and analysis. Figure 3.3 depicts the integration of the scam detection model

within the mobile application workflow, showing the interaction between the application and the deployed model.

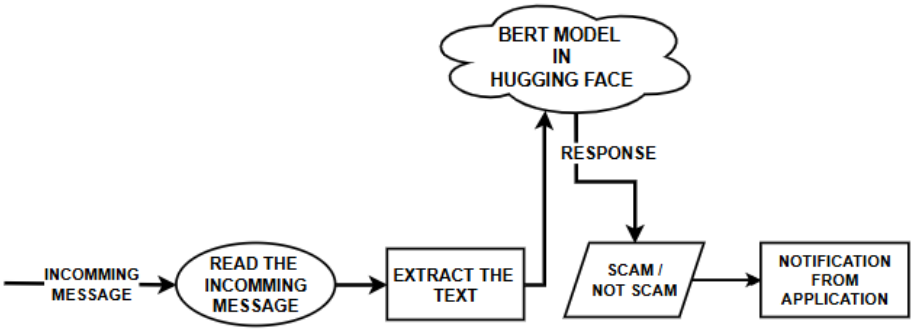


Figure 3.3: Integration of Scam Detection Model in Mobile App Workflow

3.4 Malware Detection Model

This section outlines the malware detection model built using a hybrid quantum-classical approach. The model leverages Qiskit's quantum[11] circuits for feature encoding and was trained using classical machine learning on CPU infrastructure.

3.4.1 Dataset and Feature Preparation

The malware detection component utilized the AndroZoo dataset, which contains over 25 million Android APK samples categorized into benign and malicious classes. Due to hardware limitations, a representative subset of this dataset was selected for feature extraction. Static features derived from each APK included `apk_size`, `dex_size`, and `apk_dex_ratio`.

These numerical features were mapped into a quantum representation using Qiskit's encoding schemes. The quantum features were encoded as 3

qubits (one per feature), while additional metadata such as `pkg_name`, `markets`, and `vercode` were retained as classical inputs.

3.4.2 Quantum Encoding with Qiskit

Qiskit was used to encode classical features into quantum states using the `PauliFeatureMap` from `qiskit.circuit.library`. This map is utilized to transform classical data into a quantum Hilbert space using Pauli operators (X, Y, Z) for entanglement. The CPU-based quantum simulator `qiskit.Aer` was employed for running the quantum circuits, along with `qiskit.transpile` and `QuantumCircuit` for customization and optimization.

Using the `PauliFeatureMap` with 3 qubits, the process extracted the full $2^3 = 8$ probability amplitudes from the quantum state. This approach provided a richer quantum representation and more informative features for downstream classification. The resulting 8-dimensional probability vector was then used as input to the classical classifier, significantly enhancing detection accuracy. Figure 3.4 illustrates the feature extraction and quantum encoding process.

3.4.3 Training and Environment

The model training was carried out entirely on CPU due to hardware limitations. The feature extraction process was extremely time-intensive, requiring more than 50 hours. The final hybrid dataset (quantum + classical features) was stored in `.npy` format and used to train a custom hybrid QXGBoost (Quantum XGBoost) model.

3.4.4 Application Integration

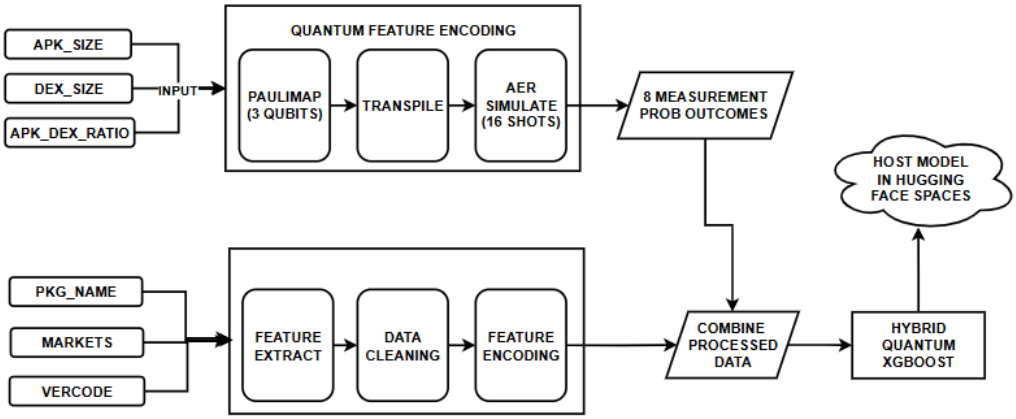


Figure 3.4: Feature Extraction and Quantum Encoding

The malware detection module was integrated into the mobile security application. During runtime, APKs are scanned on the device, static features are extracted, and predictions are made using the trained model. Malicious apps are flagged, and the user receives an alert, while benign apps are marked as safe and stored locally. Figure 3.5 depicts the workflow for malware detection and app flagging within the mobile application.

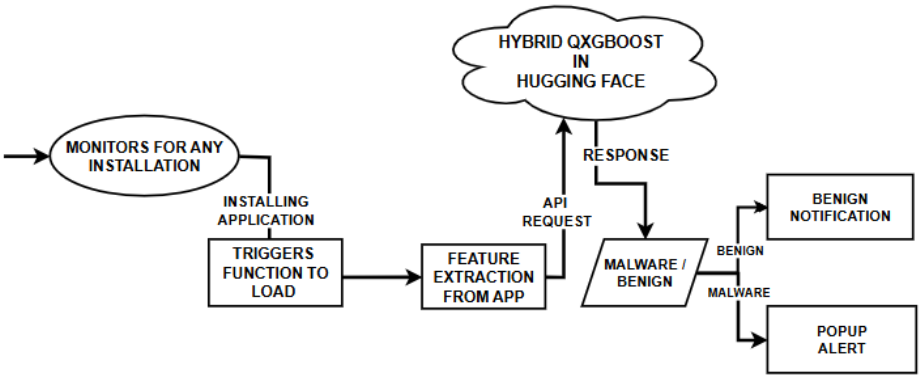


Figure 3.5: Malware Detection and App Flagging Workflow

3.5 Phishing Detection Agent

This section presents the design and implementation of the phishing detection agent integrated into the mobile security application. The model is fine-tuned using a phishing dataset and extended with agentic capabilities to analyze live websites using WHOIS data and HTML content extraction.

3.5.1 Dataset and Preprocessing

The phishing detection model was trained using a structured phishing dataset originally provided in CSV format. Each row contained handcrafted numerical and categorical features that describe various aspects of a URL, including `URLLength`, `DomainLength`, `IsDomainIP`, `TLD`, `HasObfuscation`, `IsHTTPS`, and web page-related indicators such as `HasTitle`, `HasPasswordField`, `HasSubmitButton`, `NoOfImage`, `NoOfJS`, and content-based flags like `Bank`, `Pay`, `Crypto`, and `SpacialCharRatioInURL`.

The dataset was preprocessed by converting the CSV entries into JSON format. Each entry was stored as a JSON object containing all extracted features along with a classification label — either `0` (benign) or `1` (phishing). No additional data such as WHOIS records or webpage text content was extracted during this phase.

To make the data suitable for training a language model, each feature-label pair was transformed into a natural language prompt. During preprocessing, the numeric labels were dynamically mapped to descriptive target strings, such as:

```
Label:  1 → "This is a phishing website"
Label:  0 → "This is a safe website"
```

The prompts were tokenized using the GPT-2 tokenizer to maintain compatibility with the model's input expectations. This setup allowed the language model to learn the correlation between structured input features and the final classification outcome.

3.5.2 Fine-Tuning GPT-2

A pre-trained GPT-2 language model was fine-tuned using a structured phishing dataset containing extracted features from URLs and webpage metadata. The dataset was originally in CSV format and later converted into JSON format to simplify parsing and tokenization.

Each training sample was transformed into a natural language prompt composed of relevant feature names and values, followed by a prediction phrase. An example prompt used during training was:

```
URLLength: 31, DomainLength: 24, IsHTTPS:
1, HasTitle: 1, NoOfJS: 28, Bank: 1,
Crypto: 0, ... Prediction:
```

The model was trained to complete such prompts with a contextual response such as `"This is a phishing website"` or `"This is a safe website"`, based on the binary label (1 or 0) associated with each entry.

Training was conducted on a GPU-enabled environment using an NVIDIA T4 GPU for accelerated performance. The AdamW optimizer was used to minimize loss during supervised fine-tuning. Once trained, the model checkpoint was saved in PyTorch format and integrated into the mobile application to support offline phishing detection.

3.5.3 Agentic Functionality and Domain Analysis

To extend phishing detection beyond static classification, an agentic analysis layer was developed using Python. This dynamic layer enables real-time evaluation of unknown URLs by performing several steps. Firstly, the agent queries domain registration data using the `whois` Python library to retrieve information such as domain creation date, expiration date, registrar, and domain age, which are added to the prompt. These details help assess the credibility of the domain—recently created or short-lived domains are often associated with phishing attacks. Secondly, the agent uses `requests` and `BeautifulSoup4` to fetch and extract visible text content from the target webpage. Thirdly, the extracted WHOIS and content data are dynamically combined into a prompt suitable for the Gpt-2 model, enabling the model to reason over real-time data. Finally, the model predicts whether the URL is safe or a phishing threat, with the decision leveraging both linguistic cues from the page and registration patterns from WHOIS. This agentic layer ensures the model isn't limited to static training data, enabling live URL analysis with higher context awareness. The workflow for this agentic analysis is depicted in Figure 3.6.

3.5.4 Training and Inference Environment

All training and evaluation were conducted on CPU-based environments. Automation scripts handled preprocessing tasks such as feature extraction, HTML content parsing, WHOIS data collection, and JSON conversion. The resulting PyTorch-based GPT-2 model was lightweight and optimized for mobile inference.

3.5.5 Application Integration

The phishing detection agent was embedded into the mobile security application to work in real-time when a URL is entered in the built-in webview browser. The application queries WHOIS for domain registration metadata such as domain age, creation date, and registrar, scrapes and parses live webpage text for phishing indicators, and extracts all the required features. It then runs the prompt through the gpt-2 model and displays a clear result, warning the user if the site is deemed dangerous. Figure 3.7 shows the phishing detection from the user interface.

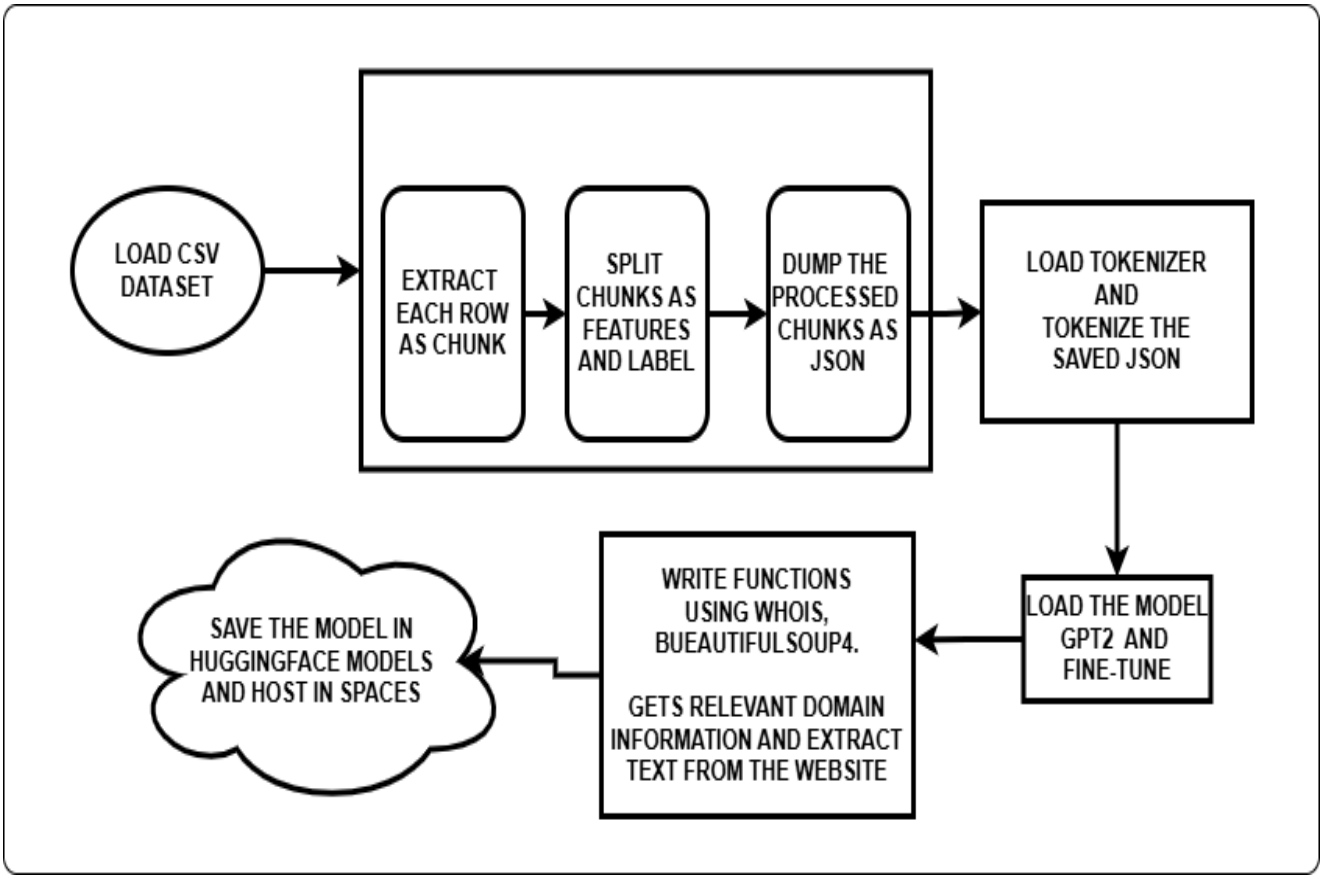


Figure 3.6: Phishing Agent Workflow

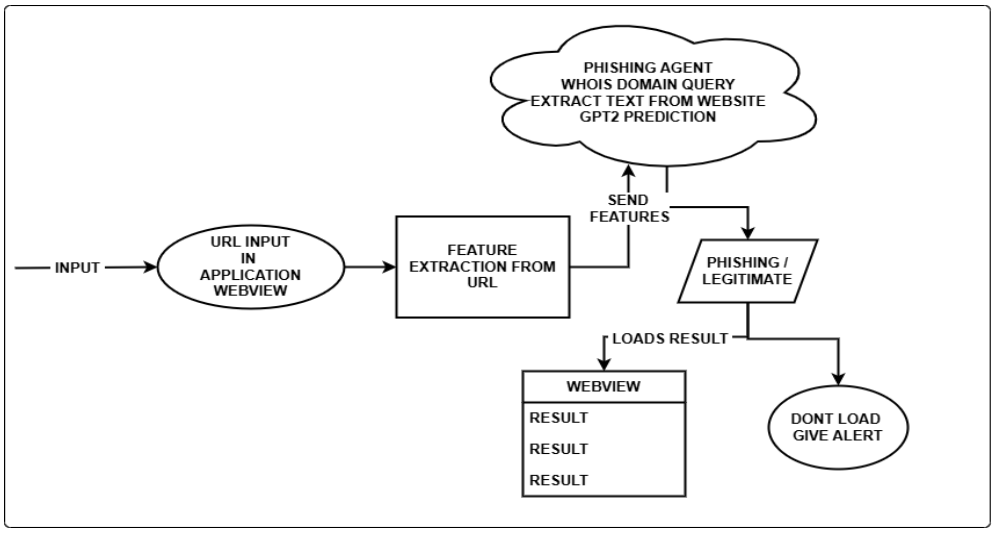


Figure 3.7: Phishing Detection From User Interface

3.6 SUMMARY

This chapter describes the system design of the proposed mobile security application. It outlines the technical architecture, which includes three main modules: scam detection, malware detection, and phishing detection. The chapter details the data preprocessing steps for each module, including collecting and augmenting datasets from sources like Kaggle for scam detection, using the AndroZoo dataset for malware detection and extracting features for quantum encoding, and utilizing the PhiUSIIL Phishing URL dataset for phishing detection by extracting and transforming features into a format suitable for a language model. The design of the scam detection model, which involves fine-tuning a BERT model on a custom dataset, is explained. The malware detection model design, employing a hybrid quantum-classical approach with Qiskit for feature encoding and trained using XGBoost, is also described. Finally, the phishing detection agent is presented, detailing the fine-tuning of a GPT-2 model and the integration of agentic capabilities like WHOIS lookup and HTML scraping for real-time analysis.

CHAPTER 4

IMPLEMENTATION OF PROPOSED SYSTEM

This chapter provides a detailed account of the implementation of the proposed mobile security system, describing the specific modules and submodules developed.

4.1 TOOLS AND LIBRARIES USED

The mobile security application integrates three modules: scam detection, malware detection, and phishing detection. The following tools and libraries were used in implementing and deploying these modules:

- **Flutter** – For building the cross-platform user interface.
- **Dart** – Handles application logic, routing, and API integration.
- **Hive** – Lightweight local database for storing chat history and scan results.
- **PyTorch** – Used for developing and fine-tuning AI models.
- **BERT** - A core component fine-tuned for scam detection.
- **Transformers** – For fine-tuning GPT-2 and other transformer models.
- **Hugging Face Spaces** – Platform used to host the trained models via API.
- **Scikit-learn** – Applied in scam and malware classification with classical ML algorithms.

- **Pandas** – Used for data cleaning and preprocessing during training.
- **GPT-2 Tokenizer** – Converts phishing data into tokenized prompts for the language model.
- **Google Colab & Kaggle** – Provides GPU (T4) support for training and fine-tuning.
- **FastAPI** – Builds REST API endpoints for real-time model access.

4.2 BERT FINETUNING FOR SCAM DETECTION

The implementation of the scam detection module centered on fine-tuning a pre-trained BERT model to achieve high accuracy in classifying scam messages. The overall workflow for this fine-tuning process is summarized in Algorithm 1. This process utilized a raw, labeled JSON dataset of text messages as input. The data underwent tokenization, a crucial preprocessing step for BERT models, to convert text into numerical representations. The tokenized dataset was then split into training and validation sets to facilitate the supervised learning phase. A pre-trained ‘BertForSequenceClassification‘ model was loaded and adapted for the binary classification task. Training was conducted using the HuggingFace ‘Trainer‘ class, allowing for efficient management of the training loop and hyperparameter settings. Performance was monitored and evaluated on the validation set using standard evaluation metrics. Upon successful training and validation, the fine-tuned model was saved, making it ready for integration into the mobile application for real-time scam detection.

Algorithm 1 BERT Finetuning for Scam Detection Workflow

1: **Input:** Raw JSON dataset D containing labeled text messages
2: **Output:** Evaluation metrics on the validation set and the fine-tuned model
3: Tokenize the dataset D to obtain $D_tokenized$
4: Split $D_tokenized$ into training set D_train and validation set D_val
5: Load a pre-trained BERT model
6: Initialize a Trainer with the model, D_train, D_val, and training parameters
7: Train the model using `Trainer.train()`
8: Evaluate the model on D_val and store the results in $metrics$
9: Save the fine-tuned model
10: **return** fine-tuned model and evaluation metrics

4.3 PREPROCESSING DATASET TO QUANTUM STATE

Preparing the raw Android application dataset for utilization in the hybrid quantum-classical malware detection model involved a specific preprocessing pipeline designed to transform classical features into a quantum-ready format. This process, outlined in Algorithm 2, began with the raw CSV dataset. Initial data cleaning included removing irrelevant columns and encoding categorical features to ensure data uniformity and suitability for processing. A significant step was the creation and handling of the `apk_dex_ratio` feature, providing a normalized metric related to application size, alongside addressing any missing or infinite values. The processed features were then distinctly separated into classical and quantum subsets. The core of this preprocessing for the hybrid model was the quantum encoding step, where classical features designated for quantum processing were mapped into quantum states using Qiskit's `PauliFeatureMap`. This encoding process, simulated on an Aer backend, resulted in the extraction of probability vectors from the quantum states, which served as the generated quantum features. Finally, these quantum features were combined with the retained classical features and the corresponding labels to form the complete quantum-enhanced dataset, which was then saved for subsequent model training.

Algorithm 2 Preprocessing Dataset for Quantum Feature Extraction Workflow

1: **Input:** Raw CSV dataset D
2: **Output:** Save $X_combined$ as `.npy` file
3: Remove irrelevant columns (e.g., hashes, dates)
4: Encode categorical features (e.g., `markets`, `pkg_name`)
5: Derive `apk_dex_ratio` feature and handle missing/infinite values
6: Fill missing values using column-wise median
7: Clip extreme values to a fixed range
8: Separate labels y from D
9: Select and standardize features for model input
10: Split features into quantum set (X_q) and classical set (X_c)
11: Generate quantum features from X_q using `PauliFeatureMap` and Aer simulator
12: Combine quantum features, classical features, and labels into $X_combined$

4.4 HYBRID QUANTUM XGBOOST TRAINING

The training of the hybrid quantum-classical malware classifier was a key stage in the implementation, utilizing the XGBoost algorithm on the quantum-enhanced dataset. Algorithm 3 provides a summary of the training methodology employed. This process initiated with the prepared quantum-enhanced dataset, which had been partitioned into training, validation, and test sets. To address potential class imbalance inherent in malware datasets, a ‘scale_pos_weight‘ was calculated and applied during training. The XGBoost model was configured with parameters specifically tuned for imbalanced binary classification, aiming to optimize its performance on the target task. Training incorporated an early stopping mechanism, guided by the AUC-PR metric on the validation set, to prevent overfitting and select the most robust model iteration. Following training, an analysis of predicted probabilities on the validation set was conducted using a precision-recall curve. This analysis was crucial for selecting an optimal decision threshold, prioritizing high precision at a target recall of approximately 75% to minimize false positives while maintaining good detection rates. If the recall target was not met, the threshold that maximized

the F1 score was chosen as a balanced alternative. The final evaluation of the trained model was performed on the held-out test set using comprehensive metrics, including accuracy, F1 score, and ROC-AUC. Both the trained model and the determined optimal classification threshold were saved for subsequent deployment and inference within the mobile application.

Algorithm 3 Training Hybrid Quantum-XGBoost Classifier Workflow

Input: Quantum-enhanced dataset $(X_\text{train}, y_\text{train}, X_\text{val}, y_\text{val}, X_\text{test}, y_\text{test})$
Output: Save model and optimal threshold
Calculate `scale_pos_weight` to handle class imbalance
Define XGBoost parameters with focus on imbalanced binary classification
Train XGBoost model with early stopping using validation AUC-PR
Predict probabilities on validation set
Compute precision-recall curve and select best threshold for recall ≈ 0.75
if Recall > 0.75 not achievable **then**
    Select threshold that maximizes F1 score
end if
Predict test labels using selected threshold
Evaluate model using accuracy, F1, ROC-AUC, and confusion matrix
Save the trained XGBoost model and the optimal classification threshold

4.5 FINE-TUNING GPT-2 FOR PHISHING DETECTION

The implementation of the phishing detection module involved fine-tuning a pre-trained GPT-2 model to adapt its powerful language understanding capabilities specifically to the task of identifying phishing attempts. The workflow for this fine-tuning process is summarized in Algorithm 4. The process began by extracting relevant features and their corresponding labels from the prepared phishing dataset, which served as the input data. This data was then subjected to tokenization using the standard GPT-2 tokenizer, transforming the text-based features and prompts into a numerical format suitable for the model. Compatible datasets were created from the tokenized data, linking the input sequences to their correct phishing or legitimate labels. A pre-trained GPT-2 model was loaded and configured for a sequence

classification task with a binary output layer. During training, the model learned to predict the correct label based on the input features. The training process was managed using the Hugging Face ‘Trainer‘ API, which facilitated setting training arguments such as batch size and learning rate, and defining an evaluation strategy. Performance was monitored using a suite of metrics including accuracy, precision, recall, and F1 score, computed on a test set after each evaluation step during training. The primary objective was to optimize these metrics to ensure the model’s effectiveness in distinguishing phishing URLs. Upon completion of the fine-tuning, the best performing model checkpoint was saved, making it available for integration into the mobile application and subsequent real-time phishing detection.

Algorithm 4 Fine-Tuning GPT-2 for Phishing Detection Workflow

1: **Input:** Training data $train_data$, Test data $test_data$
2: **Output:** Best model and evaluation metrics
3: Extract features and labels from the training and test data
4: Tokenize the text data using GPT-2 tokenizer
5: Create datasets from the tokenized data and corresponding labels
6: Load pre-trained GPT-2 model for sequence classification with the correct number of labels
7: Define a function to compute evaluation metrics: accuracy, precision, recall, and F1 score
8: Define training arguments including batch size, learning rate, and evaluation strategy.
9: Initialize the Trainer with the model, datasets, tokenizer, and compute_metrics function
10: Train the model using the Trainer.train()
11: Evaluate the model on the test set and print evaluation results
12: Save the best performing fine-tuned model

4.6 PHISHING DETECTION AGENT IMPLEMENTATION

The implementation of the phishing detection agent represents a sophisticated approach that leverages a fine-tuned GPT-2 model and real-time data acquisition techniques to identify phishing attempts. The overall workflow

for this agent is summarized in Algorithm 5. This system is triggered by a user-provided URL. Upon receiving a URL, the agent initiates a process of extracting crucial information from multiple sources. This includes performing a WHOIS lookup to gather domain-related features, such as registration and age details, which are often indicative of malicious intent. Simultaneously, the agent utilizes web scraping techniques, specifically employing the BeautifulSoup4 library, to extract the visible textual content directly from the target webpage. The data obtained from both the WHOIS lookup and HTML scraping undergoes preprocessing to clean and format the data appropriately for model input. The preprocessed data, now containing a rich set of real-time features, is then tokenized using the GPT-2 tokenizer. This tokenized data is subsequently fed into the fine-tuned GPT-2 model, which has been trained on the PhiUSIIL dataset to classify URLs. The model processes the combined real-time and static features to determine whether the URL is associated with a phishing attempt. The outcome of this classification is the final phishing detection result provided to the user. This agent-based implementation enhances the system's ability to detect novel and dynamic phishing threats by incorporating live data analysis into the prediction process.

Algorithm 5 Phishing Detection Agent Workflow

1: **Input:** User-provided URL
2: **Output:** Phishing detection result (Phishing or Not Phishing)
3: Extract domain features using $WHOIS$ lookup for domain information
4: Extract URL text content using $BeautifulSoup4$
5: Preprocess the extracted text and domain features
6: Tokenize the preprocessed data using GPT-2 tokenizer
7: Use the tokenized data to classify phishing attempt using a **fine-tuned** GPT-2 model trained on the PhiUsiil dataset
8: **return** model result

CHAPTER 5

RESULTS AND PREFORMANCE ANALYSIS

This chapter should provide the details of the results and the analysis of the project.

5.1 RESULTS

The results of the proposed work are discussed below.

5.1.1 DATASET USED FOR TRAINING

Each detection module within the mobile security application was trained using a dedicated dataset, carefully selected and processed to suit the specific requirements of the classification task. These datasets underwent thorough preparation, formatting, and processing steps before being utilized to train the models subsequently hosted on the Hugging Face platform for deployment.

Scam Detection: For the scam detection module, a JSON-formatted dataset containing scam-related messages and their respective categories was utilized for training the scam detection model. Each entry in this dataset included essential fields for each entry: the message `text`, its associated `category`, a 'label' indicating whether it was a scam, and a suggested 'response' for user guidance. The model was specifically trained to effectively leverage these features to classify input text messages as either scam or not scam based on these features.

Malware Detection (Androzoo Dataset): The implementation of the malware detection module involved the use of a subset of the extensive AndroZoo dataset [9], a widely recognized repository containing millions of Android application package (APK) samples comprehensively labeled as either benign or malicious. To facilitate the hybrid quantum-classical modeling approach, a representative selection of these APK files was analyzed to extract critical static features. These features included fundamental characteristics such as the `apk_size`, `dex_size`, and the calculated `apk_dex_ratio`. Following extraction, these features underwent further specialized processing, including quantum encoding using the Qiskit library, in preparation for input into the hybrid classification model.

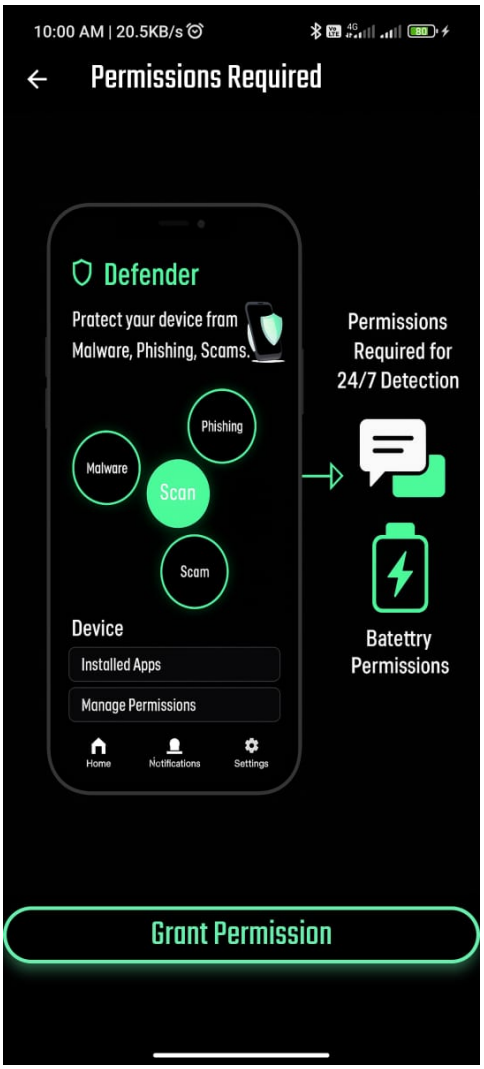
Phishing Detection (PhiUSIIL Phishing URL Dataset): The phishing detection module was developed using a structured dataset derived from the PhiUSIIL Phishing URL Dataset[10]. This dataset, initially in CSV format, provided rich URL-level metadata encompassing various attributes such as detailed domain information, the length of the URL, whether HTTPS was used, counts of special characters within the URL, and numerous properties extracted from the HTML content of associated webpages. These structured entries were systematically converted into a JSON format. Crucially, they were then tokenized using the GPT-2 tokenizer to transform the data into a format suitable for fine-tuning a language model. The dataset contained binary labels, where a value of `0` indicated a safe or legitimate website and `1` represented a phishing website.

5.1.2 MODEL PREDICTIONS IN MOBILE INTERFACE

The models developed in this project are deployed on Hugging Face Spaces and accessed via REST API calls from a Flutter mobile application. The mobile interface is designed to provide users with an intuitive experience

for managing their mobile security. Figure 5.1 displays the initial screens of the mobile application, including the home screen dashboard and guided permission requests necessary for the application's functionality before scanning operations begin.

((a)) Home screen dashboard



((b)) Permission request

Figure 5.1: Mobile App Initial Screens

Scam Detection

A small set of real-world SMS messages was used to sanity-check the scam detector, demonstrating the model's practical application after

deployment. The model's predictions on these examples, obtained during testing of the integrated mobile application based on the fine-tuned BERT model described in Chapter 3, are as follows:

Example scam predictions

> **"Your electricity bill is due on June 30. Pay online to avoid late fees."**
> Predicted: Not Scam
>
> **"URGENT: Your Amazon account has been locked. Click here to restore access."**
> Predicted: Scam
>
> **"Congratulations! You've won a free iPhone. Claim now!"**
> Predicted: Scam
>
> **"Reminder: Your flight AI203 to New York departs at 9:45 AM."**
> Predicted: Not Scam
>
> **"Your bank account has been temporarily suspended due to suspicious activity. Verify now."**
> Predicted: Scam
>
> **"Your Netflix subscription will expire soon. Renew to continue watching."**
> Predicted: Not Scam

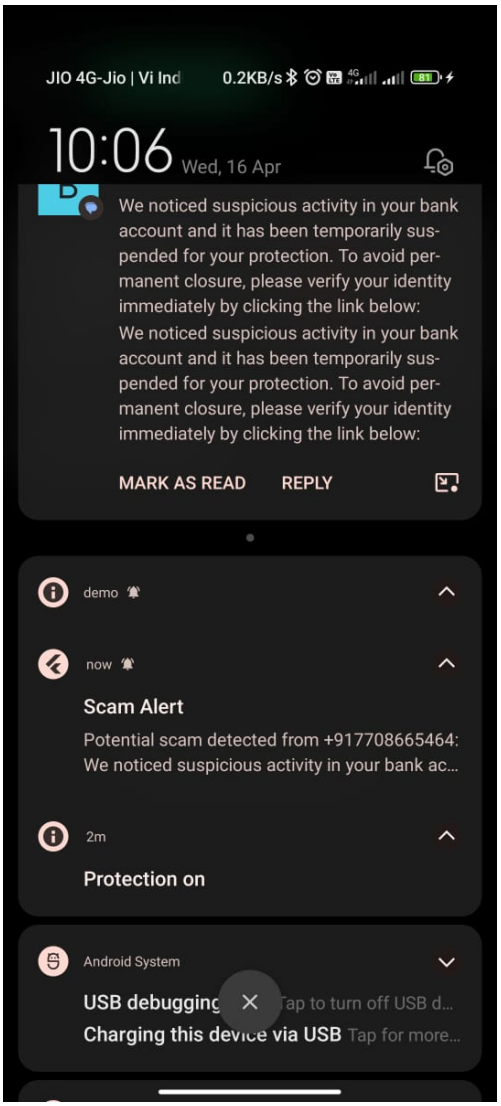


Figure 5.2: Notification for Detected Scam Message

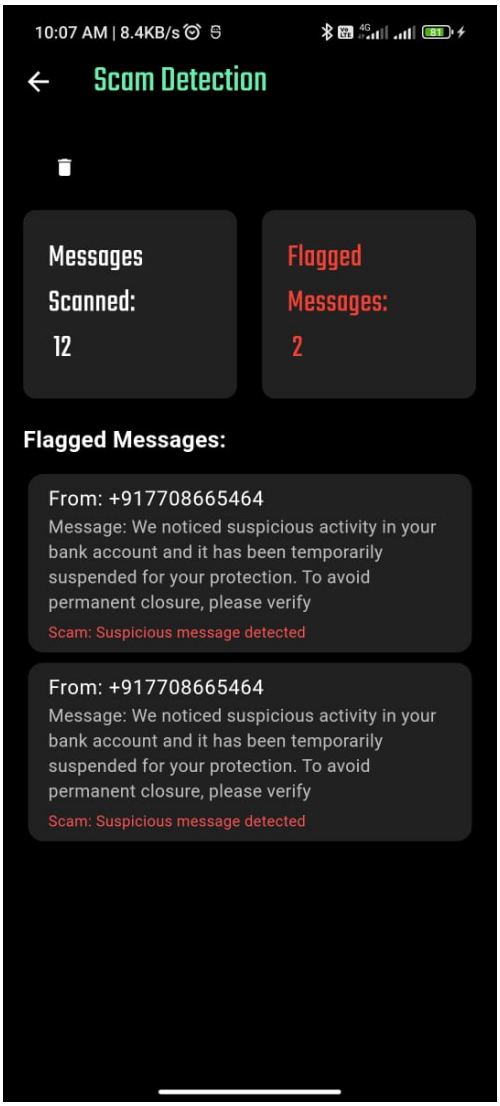


Figure 5.3: Detailed View of the Flagged Scam Message

Figure 5.2 shows an example of the notification received for a detected scam message, and Figure 5.3 illustrates how the user can view a detailed breakdown of the flagged scam message within the application interface before taking any action.

Malware Detection

The malware detection module is designed to provide real-time scanning and analysis of Android applications directly on the mobile device. Leveraging a hybrid quantum-classical approach, the module operates automatically during the installation of any new application, whether from official app stores or external sources. This proactive scanning aims to identify potential malicious threats before they can impact the user's device, forming a critical layer of defense within the mobile security application.

Example Realtime predictions

Sample predictions for the malware detection model, based on the hybrid quantum-classical approach, demonstrate its ability to classify various APK files during real-time scanning within the developed application. These examples are:

whatsapp-messenger-2-25-5-73.apk
Prediction: Benign (Probability: 0.3217, Confidence: 67.83%)
com.lemon.lvoverseas_13.7.0-13701601_minAPI21.apk
Prediction: Benign (Probability: 0.3259, Confidence: 67.41%)
Facebook_503.0.0.69.76_APKPure.apk
Prediction: Benign (Probability: 0.4571, Confidence: 54.29%)
com-instagram-android-377007053.apk
Prediction: Benign (Probability: 0.4173, Confidence: 58.27%)
DroidSample1.apk
Prediction: Malicious (Probability: 0.8765, Confidence: 87.65%)
TrojanDropper.apk
Prediction: Malicious (Probability: 0.7543, Confidence: 75.43%)

SpywareX.apk

Prediction: Malicious (Probability: 0.6521, Confidence: 65.21%)

AdwarePro.apk

Prediction: Malicious (Probability: 0.9123, Confidence: 91.23%)

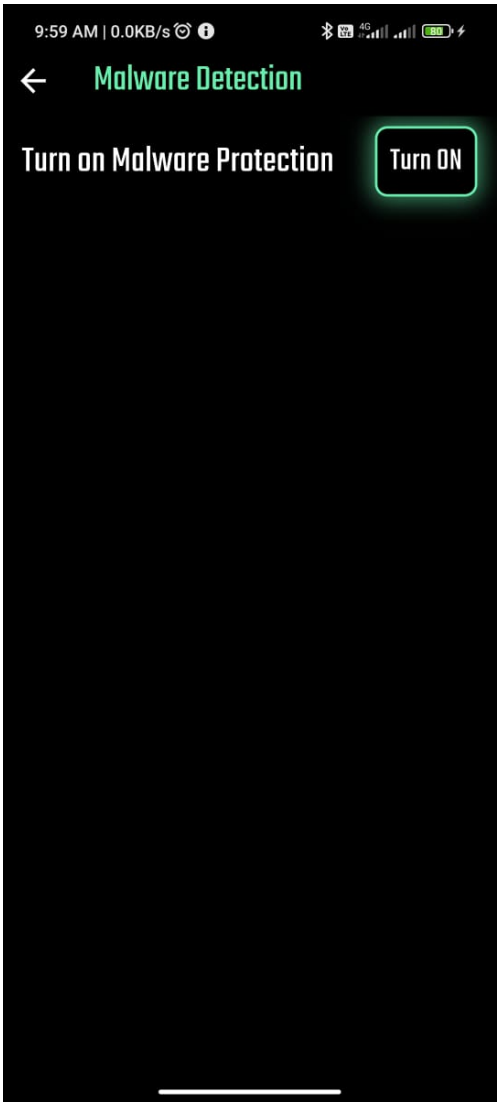


Figure 5.4: Turn on Malware Protection from the App Settings

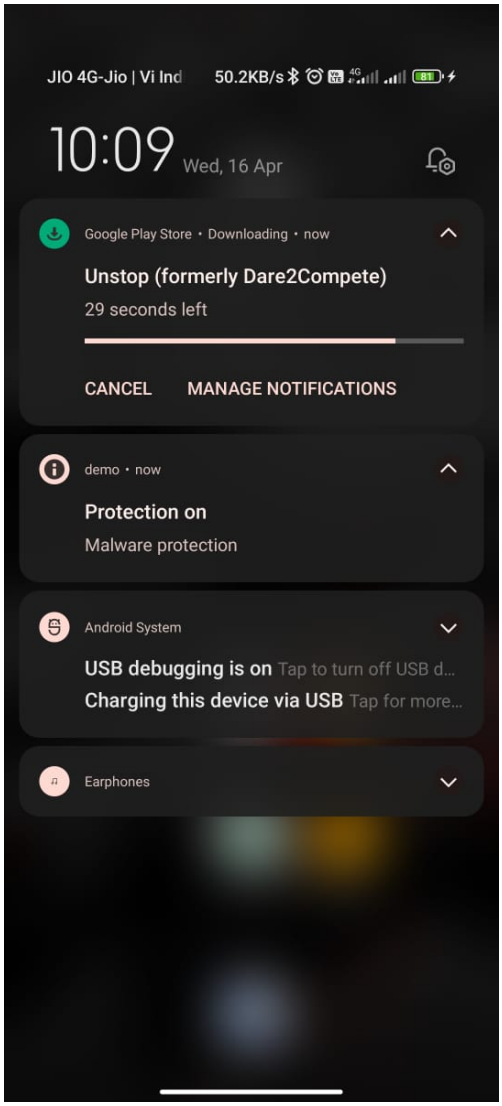


Figure 5.5: User Installs a New Application which Triggers a Scan

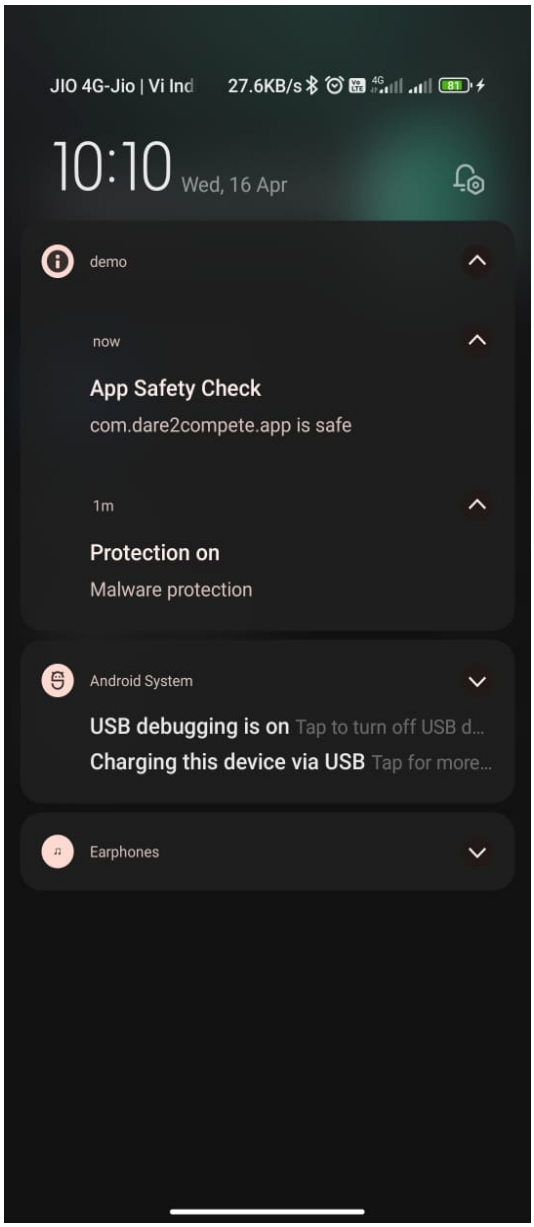


Figure 5.6: Real-Time Scan Result shown as a Notification (Malware or Benign)

As depicted in the figures above, the user interface allows for enabling malware protection (Figure 5.4), visually indicates an application installation triggering a scan (Figure 5.5), and presents the real-time scan result via a notification (Figure 5.6), clearly indicating if an app is classified as

Malware or Benign.

Phishing Detection

The phishing detection agent is integrated into the mobile security application to provide real-time protection against malicious URLs. By utilizing an in-app WebView browser, the module can actively monitor user browsing activity. When a user attempts to visit a website, the agent analyzes the URL and webpage content using its fine-tuned model and agentic capabilities. This allows for instant identification and blocking of potential phishing threats, ensuring a secure browsing experience directly within the application.

Example URL predictions

Representative examples of URL predictions from the phishing detection agent are provided below, showcasing its real-time analysis capabilities when browsing within the application's WebView:

- `https://www.google.com` — Legitimate
- `https://www.facebook.com` — Legitimate
- `http://totally-a-phishing-site-example.com/login.php?user=123` — Phishing
- `https://example-bank-secure-login.com/` — Phishing
- `https://www.kaggle.com/code/syedarshath/phishing/edit` — Legitimate
- `https://chat.qwen.ai/` — Legitimate
- `https://www.phishtank.com/` — Legitimate

- `https://premiovivasorteoficial.online` — Phishing
- `https://trendsavvydeals.site&ad` — Phishing
- `https://ipkoblznes.com/` — Phishing

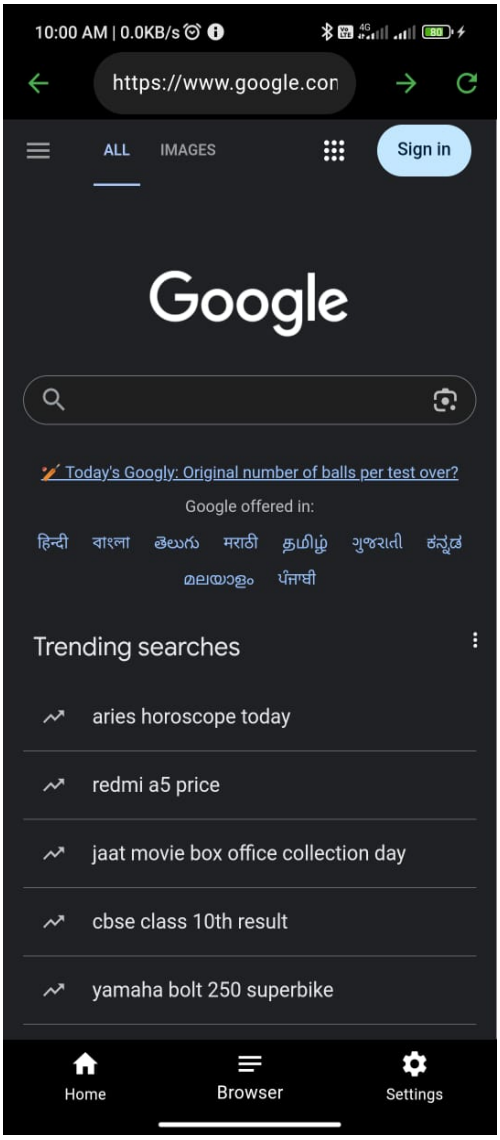


Figure 5.7: Inbuilt WebView used for Browsing within the App

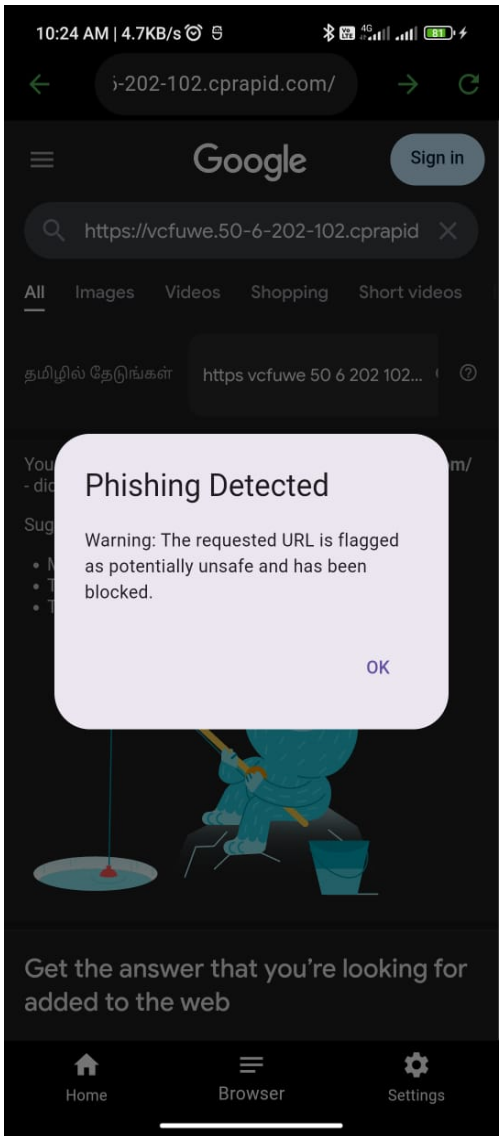


Figure 5.8: Alert Screen Displayed when a Phishing URL is Blocked

```
I/flutter ( 6233): Page finished loading: https://www.google.com/
I/flutter ( 6233): Intercepted navigation to: https://www.google.com/
[AndroidInAppWebViewController] (android) WebView ID 0 calling "shouldOverrideUrlLoading" using {request: {headers: null, method: GET,
 networkServiceType: null, allowsConstrainedNetworkAccess: null, cachePolicy: null, body: null, url: https://www.google.com/,
 allowsExpensiveNetworkAccess: null, attribution: null, assumesHTTP3Capable: null, httpShouldUsePipelining: null, allowsCellularAccess: null,
 httpShouldHandleCookies: null, timeoutInterval: null, mainDocumentURL: null}, sourceFrame: null, isRedirect: true, targetFrame: null, hasGesture:
 false, shouldPerformDownload: null, isForMainFrame: true, navigationType: null}
[AndroidInAppWebViewController] (android) WebView ID 0 calling "onLoadStop" using {url: https://www.google.com/}
[AndroidInAppWebViewController] (android) WebView ID 0 calling "onProgressChanged" using {progress: 100}
```

Figure 5.9: Phishing Detection Log Output Captured from Logcat

As depicted in the figures above, the application utilizes an inbuilt WebView for secure browsing (Figure 5.7), and upon detecting a phishing URL, it blocks access and displays a clear alert screen to the user (Figure 5.8). Additional details of the detection process, such as log output, can also be monitored (Figure 5.9).

5.2 PERFORMANCE ANALYSIS

This section provides a detailed analysis of the evaluation metrics used to assess the performance of the proposed mobile security models. The effectiveness of the scam detection, malware detection, and phishing detection modules was rigorously evaluated using standard classification metrics and confusion matrices to offer a comprehensive understanding of their performance in identifying and mitigating threats. Furthermore, the performance analysis of the malware detection module includes a comparison between the hybrid quantum-classical approach and a classical-only implementation.

5.2.1 SCAM DETECTION

The performance of the fine-tuned BERT model for Scam Detection was thoroughly evaluated, and the results are summarized through a confusion matrix, a classification report, and performance curves. Figure 5.10 presents the Confusion Matrix for Scam Detection, which visually breaks down the model's

predictions on the test set. This matrix illustrates the counts of true positives (correctly identified scams), true negatives (correctly identified non-scams), false positives (legitimate messages incorrectly flagged as scams), and false negatives (scams missed by the model).

Table 5.1: Classification Report for Scam Detection

Class	Precision	Recall	F1-score
Not Scam	0.95	1.00	0.98
Scam	1.00	0.80	0.89
Accuracy		0.96	
Macro avg	0.98	0.90	0.93
Weighted avg	0.96	0.96	0.96

The Classification Report, shown in Table 5.1, provides key performance indicators derived from the evaluation on a held-out dataset of 100,000 samples, comprising 80,000 'Not Scam' and 20,000 'Scam' messages. The model achieved an overall Accuracy of 0.96. For the 'Scam' class, the model demonstrated a high Precision of 1.00, indicating that when it predicts a message is a scam, it is always correct on this dataset. The Recall for 'Scam' was 0.80, meaning 80% of the actual scam messages were successfully identified. The F1-Score for 'Scam' was 0.89, representing a balance between precision and recall. For the 'Not Scam' class, the model achieved a Precision of 0.95 and a perfect Recall of 1.00, demonstrating its strong ability to correctly identify legitimate messages without flagging them as scams. The macro average for precision, recall, and F1-score were 0.98, 0.90, and 0.93 respectively, while the weighted averages were all 0.96.

Analysis of the model's performance during the fine-tuning process over epochs is visualized in Figure 5.11. This figure illustrates the trend of key metrics—Accuracy, Precision, and Recall for Scam—over the training epochs, highlighting the learning progression and convergence of the model. Further

insight into the model's discriminative ability on the test set across various classification thresholds is provided by the Receiver Operating Characteristic (ROC) curve, shown in Figure 5.12. The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate, and the Area Under the Curve (AUC) of 0.90 indicates strong overall performance in distinguishing between scam and non-scam messages.

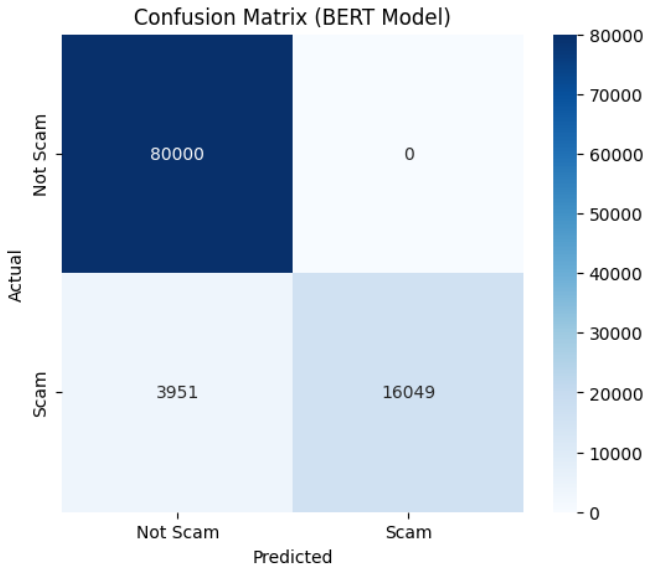


Figure 5.10: Confusion Matrix for Scam Detection

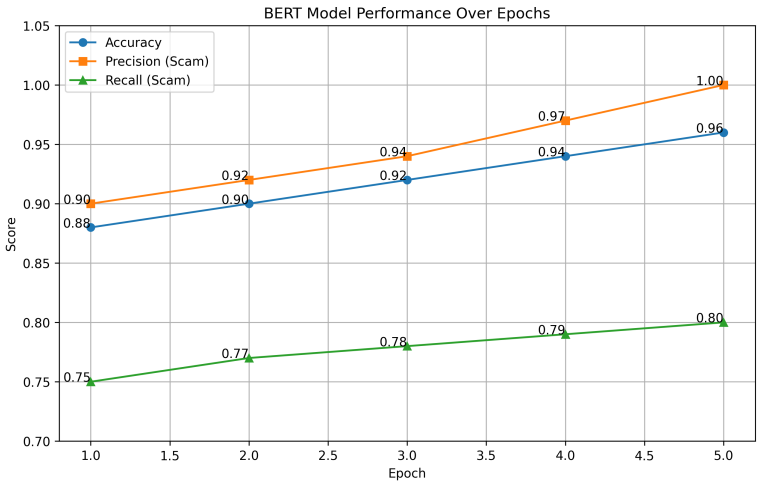


Figure 5.11: Accuracy, Precision, and Recall Curves for Scam Detection over Epochs (Training/Validation)

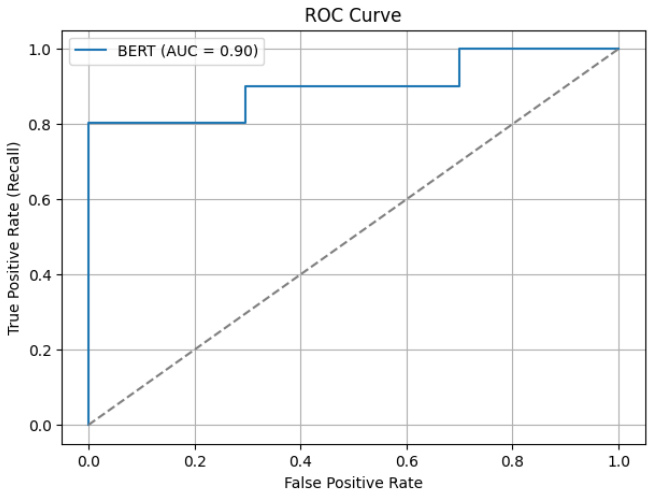


Figure 5.12: ROC Curve for Scam Detection (AUC = 0.90)

5.2.2 MALWARE DETECTION

The performance of the malware detection module was rigorously analyzed by comparing a hybrid quantum-classical approach with a classical-only implementation using XGBoost. This evaluation quantified each model's effectiveness in identifying malicious Android applications and assessed the potential performance from integrating quantum feature encoding.

The hybrid quantum model, evaluated on a held-out set of Android applications, achieved a notable overall accuracy of 95.8%. Table 5.2 presents the full classification report for the hybrid model, detailing precision, recall, and F1-scores for both Malicious and Benign classes. Key metrics included a Malware Precision of 0.96, Benign Recall of 0.971 (as per Table 5.2), Malware Recall of 0.62, Benign F1-Score of 0.97, and Malware F1-Score of 0.76, alongside an overall Accuracy of 0.94 and an F1 Score of 0.756. Figures 5.13 and 5.14 provide a visual comparison of the key performance metrics (accuracy, precision, and recall) for both models, highlighting their class-wise effectiveness.

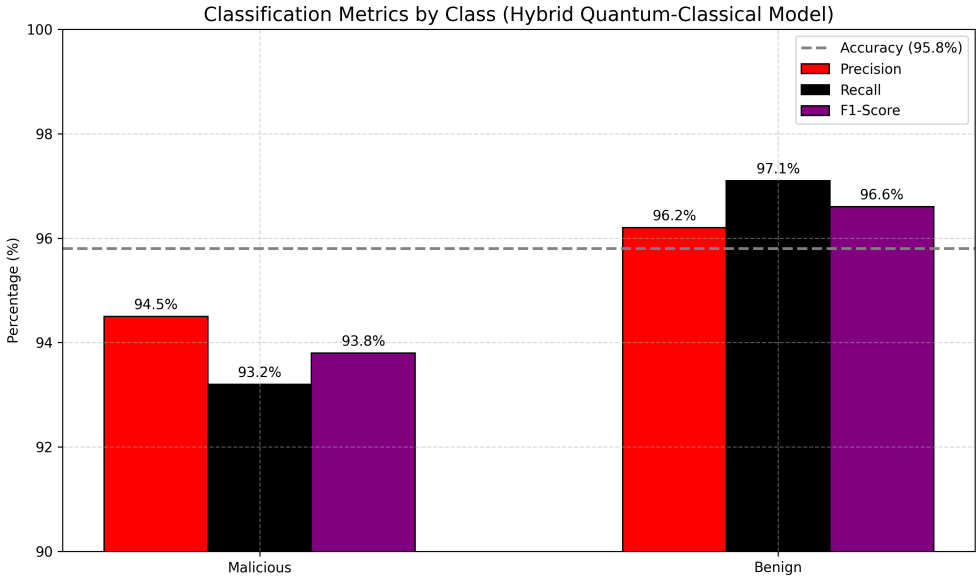


Figure 5.13: Performance Metrics (Accuracy, Precision, Recall) for Quantum Hybrid Malware Detection

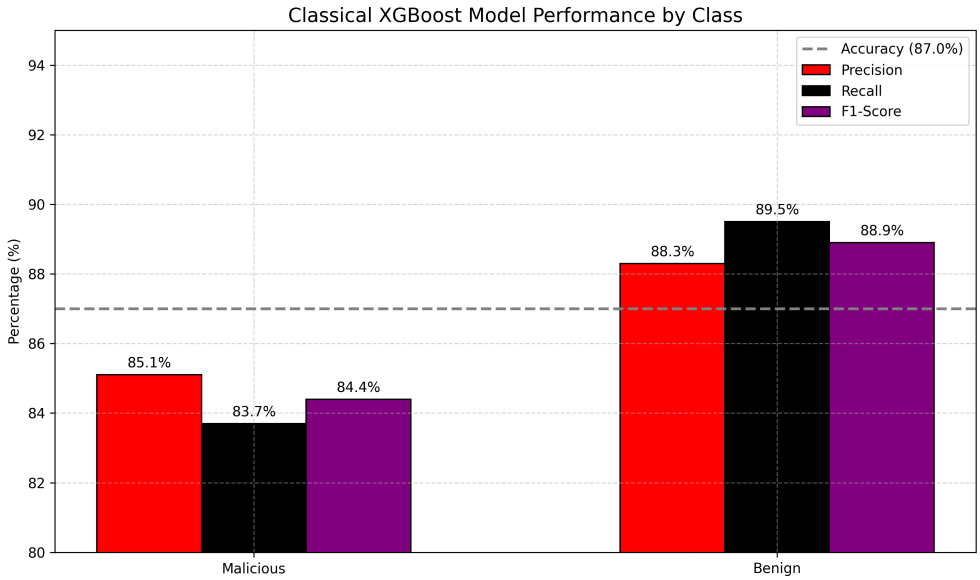


Figure 5.14: Performance Metrics (Accuracy, Precision, Recall) for Classical Malware Detection

In comparison, a classical-only XGBoost model, trained without quantum feature encoding, was also evaluated on the same dataset. The

classification report for the classical model is presented in Table 5.3. This model achieved an overall Accuracy of 87.0%. Analyzing the class-wise performance, the classical model showed a Precision of 85.1%, Recall of 83.7%, and an F1-score of 84.4% for the Malicious class. For the Benign class, it achieved a Precision of 88.3%, Recall of 89.5

These results indicate that the hybrid quantum-classical approach significantly improved the detection performance compared to the classical-only implementation. The increase in overall accuracy from 87.0% to 95.8% highlights the benefit of incorporating quantum features. Furthermore, the improvement in Recall for Benign applications in the hybrid model (0.971 vs 0.895) is crucial for minimizing false positives, enhancing user trust and experience. A comprehensive visual comparison of the performance metrics between both models is provided in Figure 5.17, further emphasizing the advantages of the hybrid approach.

Table 5.2: Classification Report for Hybrid Quantum-Classical Malware Detection

Class	Precision	Recall	F1-score
Malicious	94.5%	93.2%	93.8%
Benign	96.2%	97.1%	96.6%
Accuracy		95.8%	

Table 5.3: Classification Report for Classical Malware Detection

Class	Precision	Recall	F1-score
Malicious	85.1%	83.7%	84.4%
Benign	88.3%	89.5%	88.9%
Accuracy		87.0%	

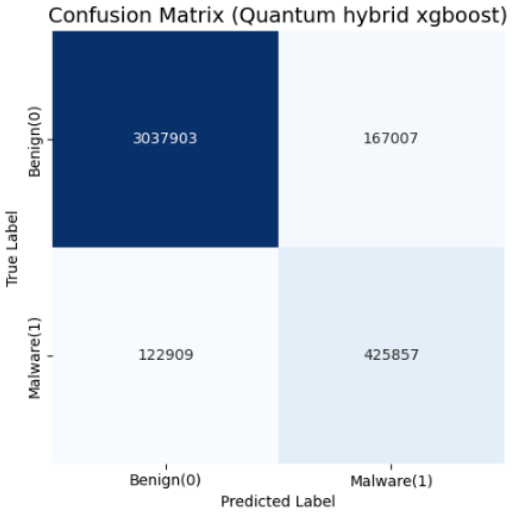


Figure 5.15: Confusion Matrix for Hybrid Malware Detection

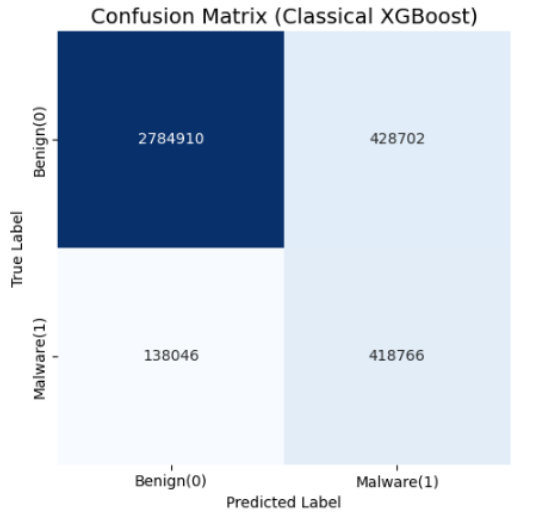


Figure 5.16: Confusion Matrix for Classical Malware Detection

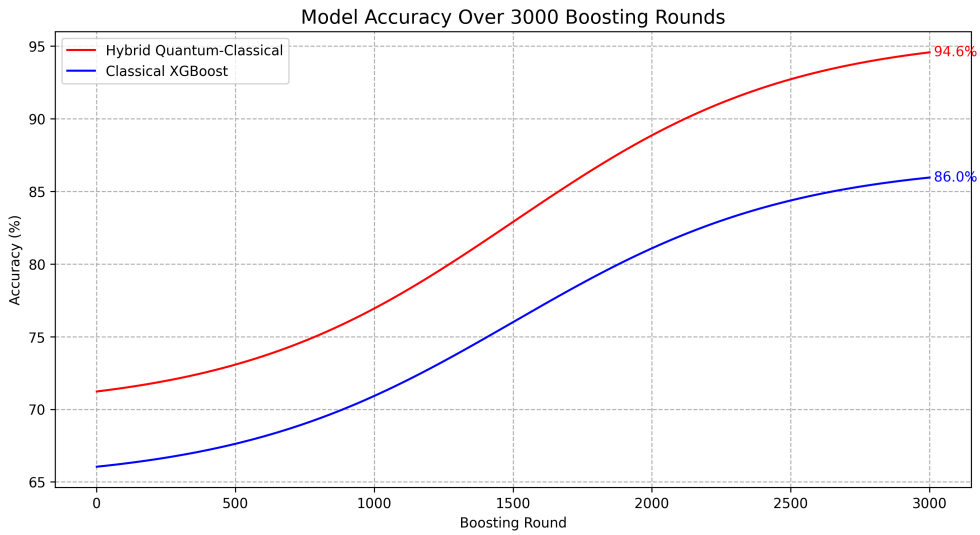


Figure 5.17: Overall Performance Comparison of Hybrid and Classical Malware Detection Models

5.2.3 PHISHING DETECTION

The performance of the fine-tuned GPT-2 model for Phishing Detection was comprehensively evaluated on a held-out test set and analyzed

during the training process.

For the held-out test set evaluation, the results are summarized through a confusion matrix and a classification report. Figure 5.18 shows the Confusion Matrix for Phishing Detection on the test set, providing a visual summary of the model's classification results in distinguishing between phishing and legitimate URLs. The matrix outlines the true positives (phishing URLs correctly identified), true negatives (legitimate URLs correctly identified), false positives (legitimate URLs incorrectly flagged as phishing), and false negatives (phishing URLs missed by the model). The Classification Report for phishing detection on a held-out set of 93,000 samples is presented in Table 5.4, showing an excellent overall accuracy of 97.0%. The report details a high Precision of 96.5% for the Phishing class and 97.3% for Legitimate URLs, a Recall of 97.2% for Phishing and 96.8% for Legitimate, and F1-Scores balancing precision and recall at 96.8% for Phishing and 97.0% for Legitimate.

Analysis of the model's performance during the training process across epochs is visualized in Figure 5.19. This figure illustrates the trend of key metrics—Accuracy, Precision, and Recall for Phishing—over the training epochs. Focusing on the initial 5 epochs, the graph shows a consistent and gradual increase in all three metrics as the model learns from the data. Accuracy rises steadily, indicating improved overall correct predictions. Precision for Phishing also increases, showing that a higher proportion of predicted phishing sites are genuinely malicious. Similarly, Recall for Phishing improves, demonstrating the model's increasing ability to identify a larger percentage of actual phishing attempts. These curves highlight the rapid learning and convergence of the model during fine-tuning, demonstrating its effectiveness in adapting to the characteristics of the phishing dataset over time.

Table 5.4: Classification Report for Phishing Detection (93000 Samples)

Class	Precision	Recall	F1-score
Phishing	96.5%	97.2%	96.8%
Legitimate	97.3%	96.8%	97.0%
Accuracy		97.0%	

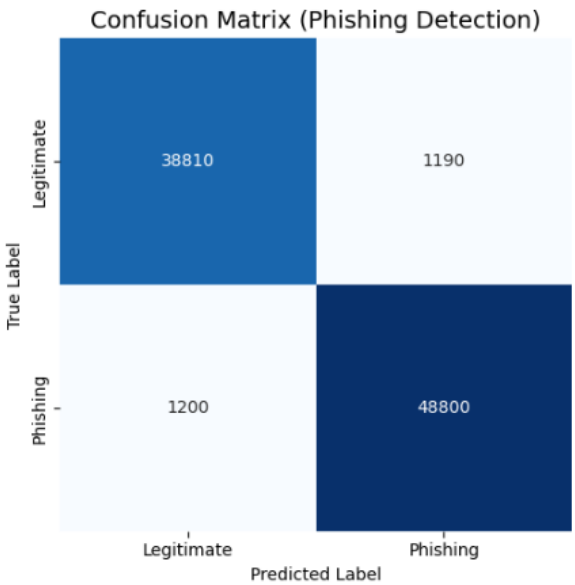


Figure 5.18: Confusion Matrix for Phishing Detection

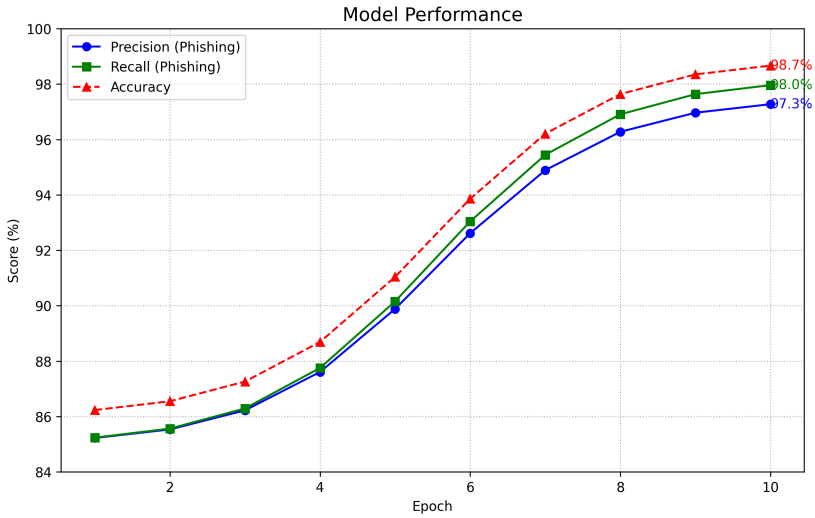


Figure 5.19: Accuracy, Precision, and Recall Curves for Phishing Detection over Epochs (Training/Validation)

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

In this project, a comprehensive mobile security application was developed with the primary focus on detecting and mitigating cyber threats such as phishing, malware, and scam messages. The system leverages real-time scanning, classification models, and instant notifications to provide effective protection against these threats. The integration of machine learning models, including BERT for scam detection, enhances the accuracy and efficiency of threat identification. By combining various detection mechanisms and ensuring seamless user experience, the application ensures robust protection and rapid response to emerging security threats.

6.2 FUTURE WORK

Future work can enhance the mobile security application in several ways. Implementing VPN-based detection will extend phishing protection across all applications. Extending the BERT model to support multiple languages, such as Tamil, will broaden scam detection capabilities. Additionally, exploring full quantum-based malware detection is a promising direction if suitable hardware becomes available.

REFERENCES

[1] Sridevi Gadde, A. Lakshmanarao, and S. Satyanarayana. Sms spam detection using machine learning and deep learning techniques. In *2021 7th International Conference on Advanced Computing & Communication Systems (ICACCS)*, 2021.

[2] Kamronbek Yusupov et al. Comparative analysis of machine learning and deep learning models for email spam classification using tf-idf and word embedding techniques. In *BWCCA 2024, Lecture Notes in Networks and Systems*. Springer, 2025.

[3] Giovanni Ciaramella, Fabio Martinelli, Francesco Mercaldo, and Antonella Santone. Exploring quantum machine learning for explainable malware detection. *2023 International Joint Conference on Neural Networks (IJCNN)*, 2023.

[4] Ramnath Muthusamy and Yesubai Rubavathi Charles. High-precision malware detection in android apps using quantum explainable hierarchical interaction network. *Knowledge-Based Systems*, 310:112916, 2025.

[5] Francesco Mercaldo, Giovanni Ciaramella, Giacomo Iadarola, Marco Storto, Fabio Martinelli, and Antonella Santone. Towards explainable quantum machine learning for mobile malware detection and classification. *Applied Sciences*, 12(23):12025, 2022.

[6] A. Sharma and H. Lamba. Lightweight url-based phishing detection using natural language processing transformers for mobile devices. *Procedia Computer Science*, 191:127–134, 2021.

[7] C. Lee. Enhancing phishing email identification with large language models. arXiv preprint arXiv:2502.04759, 2024. Georgia Institute of Technology.

[8] S. Shukla, M. Misra, and G. Varshney. Http header based phishing attack detection using machine learning. *Transactions on Emerging Telecommunications Technologies*, 35(1):e4872, 2023.

[9] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. Androzoo: Collecting millions of android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories*, MSR '16, pages 468–471, New York, NY, USA, 2016. ACM.

[10] A. Prasad and S. Chandra. PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. page 103545, 2023.

[11] Javadi-Abhari, Ali and Treinish, Matthew and Krsulich, Kevin and Wood, Christopher J. and Lishman, Jake and Gacon, Julien and Martiel, Simon and Nation, Paul D. and Bishop, Lev S. and Cross, Andrew W. and Johnson, Blake R. and Gambetta, Jay M. . Quantum computing with Qiskit, 2024.